SCALE FOR PROJECT RAG AGAINST THE MACHINE (/PROJECTS/RAG-AGAINST-THE-MACHINE)

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere-v2.42belgium.be:vogsphere/intra-uuid-610055ea-470f-4cd
```

Introduction

- Remain polite, respectful and constructive during the evaluation.
- This project evaluates information retrieval, text chunking,
answer generation and overall RAG system design.
- Reference exam scripts are provided in `exams/scripts/`:
`exam_retrieval.sh`, `exam_answer.sh`, `exam_edge_cases.sh`
- See `exams/exam_scripts_instructions.md` for detailed corrector guidance.
- The correction is designed so the retrieval pipeline (~8 min) runs
in the background while you inspect code and ask questions.
- Every mandatory question is pass/fail. The whole mandatory part must be
validated (all questions pass) before the bonus is graded.

Guidelines

- Only grade the work present in the student's Git repository.
- Set up an evaluation workspace: clone the student repository into a
subdirectory named `student/`, and place the provided `moulinette`
binary and the `exams/` scripts alongside it.
- Run `cd student && uv sync` to install the student's dependencies.
The student's `pyproject.toml` and `uv.lock` live at the root of their
repository, so `uv sync` is expected to work from there.
- The student solution must NEVER import moulinette packages.
Verify (match imports only, not the word in comments/docstrings):
`grep -rnE "^[[:space:]]*(import|from)[[:space:]]+moulinette" student/ --include="*.py"`
- The program must not crash unexpectedly during evaluation.
- No modification of student code is allowed during correction.
- The datasets are provided with this scale. Students work only with the
public datasets; for grading, unzip the corrector-only private datasets:
`unzip data/datasets/datasets_private.zip -d data/datasets/`
These private datasets are NOT expected to be in the student's repository.
- Use flags if cheating or plagiarism is detected.

Attachments

datasets_private.zip (https://cdn.intra.42.fr/document/document/54661/datasets_private.zip)

exams.zip (https://cdn.intra.42.fr/document/document/54662/exams.zip)

subject.pdf (https://cdn.intra.42.fr/pdf/pdf/220264/en.subject.pdf)

datasets_public.zip (https://cdn.intra.42.fr/document/document/54663/datasets_public.zip)

README.md (https://cdn.intra.42.fr/document/document/54664/README.md)

vllm-0.10.1.zip (https://cdn.intra.42.fr/document/document/54665/vllm-0.10.1.zip)

moulinette.zip (https://cdn.intra.42.fr/document/document/54666/moulinette.zip)

Mandatory Part

Preliminaries & Setup

Check the following requirements:

- The Git repository is available and contains only the requested files
- Python 3.10+ is used
- uv is used as the project and package manager
- pyproject.toml and uv.lock files are present
- src/ directory exists with the main module
- a Makefile is present at the repository root
- README.md file is present
- uv sync completes without errors (run from the student repo root, i.e. `cd student && uv sync` in the evaluation workspace)
- Corrector-only step: unzip the private datasets provided with this scale: unzip data/datasets/datasets_private.zip -d data/datasets/ (the student is NOT expected to ship private datasets)

Are all setup requirements met?

Yes No

Launch Pipeline (Background)

Launch the automated retrieval pipeline in a background terminal.
This runs indexing, search, and evaluation (~8 min total).

In a separate terminal, run:
./exams/scripts/exam_retrieval.sh \
--student-path ./student \
--moulinette-path ./moulinette-ubuntu

Continue with code inspection (Q3-Q6) while this runs.
Keep this run's output (terminal log and summary.log): questions Q7 to Q10 read their results from it (docs/code Recall@5, indexing time and throughput). Do not close the terminal before Q10.

Did the pipeline launch successfully?

Yes No

Code Quality & Pydantic Models

Review code quality and data model implementation. Check ALL of the following:

Code quality:

- Data models use pydantic for validation and type safety (service or orchestration classes such as the indexer/

retriever/pipeline need not be pydantic)
- Code adheres to flake8 coding standards, and passes mypy (run `make lint`, which the Common Instructions define as flake8 + mypy)
- The Makefile exposes the required rules (install, run, debug, clean, lint)
- Exceptions are handled gracefully with try-except blocks
- Resources are properly managed (context managers or explicit close())
- Progress bars are implemented for long-running operations (tqdm)

Pydantic models (all 8 must be correctly implemented):

- MinimalSource (file_path, first_character_index, last_character_index)
- UnansweredQuestion and AnsweredQuestion
- RagDataset
- MinimalSearchResults and MinimalAnswer
- StudentSearchResults and StudentSearchResultsAndAnswer

Output format:

- Search operations output valid JSON using StudentSearchResults model
- Answer operations output valid JSON using StudentSearchResultsAndAnswer model
- Each source includes correct file_path, first_character_index, last_character_index

Does the code meet all quality standards and are all models correctly implemented?

Yes No

Chunking Strategies

Verify chunking strategies:

- At least 2 chunking strategies are implemented (e.g., Python code + markdown/text)
- The chunk size default is 2000 characters and is configurable via --max_chunk_size
- Student can explain the impact of chunk size on retrieval performance: "What happens if chunks are too small? Too large?"

Ask the student to show the code for both strategies.

Are chunking strategies properly implemented?

Yes No

Retrieval System

Verify retrieval implementation:

- At least one method (TF-IDF or BM25) is implemented
- System retrieves and ranks relevant information
- Student can explain how their chosen method works

Live demo: ask the student to run a search query and explain the results:
uv run python -m src search "How to configure OpenAI server?" --k 10

Is the retrieval system properly implemented?

Yes No

Answer Generation

Verify answer generation:

- Qwen/Qwen3-0.6B is used as the DEFAULT model

- Run: uv run python -m src answer "How to configure OpenAI server?" --k 10
- System passes retrieved context to LLM within token limits
- Answers are generated based on retrieved code/documentation

Note: The student may support additional local models beyond the default.
Check the README for documentation of supported models.
During evaluation, always test with the default model (Qwen3-0.6B).

Does answer generation work correctly with the default model?

☑ Yes ☒ No

Retrieval Performance: Docs

Check the exam_retrieval.sh results.

The script evaluates Recall@5 on the private docs dataset (100 questions).
Look for the "Docs Recall@5" line in the output.

Pass criterion: Docs Recall@5 >= 80% (the threshold stated in the subject).

Does the docs dataset meet the 80% Recall@5 threshold?

☑ Yes ☒ No

Retrieval Performance: Code

Check the exam_retrieval.sh results.

The script evaluates Recall@5 on the private code dataset (100 questions).
Look for the "Code Recall@5" line in the output.

Pass criterion: Code Recall@5 >= 50% (the threshold stated in the subject).

Does the code dataset meet the 50% Recall@5 threshold?

☑ Yes ☒ No

Retrieval Performance: Indexing Time

Check the exam_retrieval.sh results.

Test 1/4 of the script times the full indexing run against the
300s (5 min) limit stated in the subject. Look for the "Indexing"
line in the output (or the "INDEX:" line in summary.log).

Pass criterion: indexing completed in <= 300s
(reported PASSED / "INDEX: OK", not EXCEEDED or CRASHED).

Did indexing complete within the 5-minute limit?

☑ Yes ☒ No

Retrieval Performance: Throughput

Check the exam_retrieval.sh results.

Test 2/4 of the script times search over the two 100-question
datasets (200 questions total) against the 90s limit stated in
the subject. Look for the "Warm Retrieval Throughput" line
(or the "THROUGHPUT:" line in summary.log).

Pass criterion: 200 questions retrieved in <= 90s

(reported PASSED / "THROUGHPUT: OK", not EXCEEDED or CRASHED).

Did retrieval stay within the 90s throughput limit?

☑ Yes ☒ No

Answer Quality

Run the answer exam script to evaluate answer quality:

./exams/scripts/exam_answer.sh \
--student-path ./student \
--moulinette-path ./moulinette-ubuntu

The script lists questions where sources were correctly retrieved,
then you pick 3 questions. For each, the student's answer is displayed.

The mandatory Qwen/Qwen3-0.6B model has known reasoning limits, so grading
prioritizes retrieval quality and grounding over perfect final phrasing.
A satisfactory answer should:

- be coherent and understandable,
- be mostly grounded in the retrieved sources (no major hallucination),
- address the question asked. Minor incompleteness due to base-model limitations is acceptable.

Pass if at least 2 out of 3 answers are satisfactory.

Does the student pass the answer evaluation (2/3 answers ok)?

☑ Yes ☒ No

Student Understanding

Test the student's understanding of RAG concepts and their implementation.
Ask these 5 questions (in order):

1. "What is Retrieval-Augmented Generation (RAG) and why is it useful?"
2. "Walk me through your complete RAG pipeline, from raw documents to a generated answer."
3. "What is the difference between TF-IDF and BM25 as retrieval methods?"
4. "What implementation choices did you make and why? What trade-offs did you consider?"
5. "If you had more time, what would you improve in your system?"

Pass requires 4 out of 5 correct/satisfactory answers.
A satisfactory answer demonstrates genuine understanding,
not just memorized definitions.

Does the student demonstrate good understanding (4/5)?

☑ Yes ☒ No

README & Documentation

Verify README.md contains all required sections. Checklist:

1. [ ] System architecture (RAG pipeline components and interactions)
2. [ ] Chunking strategy explanation
3. [ ] Retrieval method details (algorithm and ranking mechanism)
4. [ ] Performance analysis (recall@k scores and system performance)
5. [ ] Design decisions and trade-offs
6. [ ] Example usage with clear instructions

If the student implemented bonus features, the README should also
document those experiments and their results.

Pass if at least 5 out of 6 items are present and well-documented.

Is the README comprehensive and well-documented?

☑ Yes ☒ No

System Reliability

Run the edge case exam script:

./exams/scripts/exam_edge_cases.sh --student-path ./student

This tests 4 degenerate inputs:

1. Empty query: search "" --k 10
2. Gibberish query: search "asdfghjkl" --k 10
3. k=0: answer "What is vLLM?" --k 0
4. Bad dataset path: search_dataset --dataset_path /nonexistent.json

Pass criterion: ALL 4 tests complete without Python tracebacks.
The program may print error messages or return empty results: that
is acceptable. Unhandled exceptions with tracebacks are NOT acceptable.

Does the system handle all edge cases without crashing?

☑ Yes ☒ No

Bonus Part

Bonus: Advanced RAG

Graded only once the entire mandatory part is validated.

There are exactly five bonuses, each worth one point (one star). They all
run on a CPU-only campus machine. Award one star per bonus that is
implemented AND demonstrated working (not merely described in the README):

1. Semantic embeddings used for retrieval (CPU model, e.g. all-MiniLM-L6-v2)
2. Hybrid retrieval combining the lexical and semantic rankings
3. Incremental indexing (re-index only changed files)
4. Caching of the index and/or query results
5. Local HTTP API to query the index and answer questions

Star rating = number of bonuses working (0 -> 1 star, 5 -> 5 stars).
Ask the student to demonstrate each claimed bonus.

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat Crash

▲ Concerning situation ⊘ Forbidden function 💬 Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33) | Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2) | Privacy policy (https://profile.intra.42.fr/legal/terms/5) | General term of use of the site (https://profile.intra.42.fr/legal/terms/6) | Rules of procedure (https://profile.intra.42.fr/legal/terms/4) | Terms of use for video surveillance (https://profile.intra.42.fr/legal/terms/1) | Legal notices (https://profile.intra.42.fr/legal/terms/3)